

FICHE DE RÉVISION — COMPILATION

Analyse lexicale • Grammaires algébriques • Analyse syntaxique descendante (LL(1))

Vue d'ensemble de la chaîne de compilation

Compilation = traduction d'un programme source vers un langage cible + diagnostic d'erreurs.

Texte → Analyse lexicale → Séq. de tokens → Analyse syntaxique → Arbre syntaxique

- **Outils associés** : lexicale → expressions régulières / AEF ; syntaxique → grammaires / automates à pile ; sémantique → traduction dirigée par la syntaxe.

1. Analyse lexicale

But : transformer un texte (chaîne de caractères) en une séquence de **tokens** (unités lexicales), en reconnaissant mots-clés, identificateurs, constantes, séparateurs ; en ignorant espaces/commentaires ; en alimentant la table des symboles.

Méthode pour reconnaître un token

- 1. Décrire le token par une **expression régulière** (ex. identificateur = $[a-zA-Z][a-zA-Z0-9]^*$).
- 2. Construire l'**automate à états finis (AEF)** qui reconnaît cette expression régulière.
- 3. En pratique, générateurs automatiques (Lex/Flex pour C, JLex/JFlex pour Java) : ER → AFN → AFD (détermination) → AFD minimal.

Astuce méthode / réflexe examen : pour un token donné, toujours vérifier qu'on sait (a) écrire sa définition régulière, (b) dessiner l'automate correspondant état par état, en repérant état initial et état(s) final(aux).

2. Grammaires algébriques

Définition : $G = \langle T, N, P, S \rangle$ avec T = terminaux, N = non-terminaux, $S \in N$ = axiome, $P \subseteq N \times (N \cup T)^*$ = règles de production ($X \rightarrow \omega$).

2.1 Dérivation et langage

- **Dérivation simple** $m \rightarrow m'$: on remplace une occurrence d'un non-terminal X par le membre droit d'une règle $X \rightarrow x_1 \dots x_n$.
- **Dérivation gauche/droite** : on remplace toujours le non-terminal le plus à gauche / le plus à droite.
- Notation : $A \rightarrow^* \omega$ (0 ou plusieurs dérivations), $A \rightarrow^+ \omega$ (au moins une dérivation).
- **Langage** $L(G) = \{ w \in T^* \mid S \rightarrow^+ w \}$.
- **Arbre de dérivation** : racine = axiome ; feuilles = terminaux ou ϵ (lues de gauche à droite = w) ; nœuds internes = non-terminaux ; les fils d'un nœud X sont $Y_1 \dots Y_k$ ssi $X \rightarrow Y_1 \dots Y_k$ est une règle.

2.2 Propriétés à savoir tester

Propriété	Définition / Méthode de vérification
Non-terminal accessible	$\exists \beta, \delta$ tels que $S \rightarrow^* \beta A \delta$ (A apparaît dans une forme dérivée depuis S). Sinon : inaccessible.
Non-terminal productif	$\exists w \in T^*$ tel que $A \rightarrow^* w$ (A peut engendrer un mot de terminaux). Sinon : improductif.
Grammaire réduite	Tous les non-terminaux sont à la fois accessibles ET productifs (sinon les supprimer).
Réversivité à gauche (générale)	$\exists A$ tel que $A \rightarrow^+ A\beta$ (une dérivation ramène A en tête).
Réversivité à gauche immédiate	$\exists$ une production $A \rightarrow A\beta$ directement dans la grammaire.
Grammaire ambiguë	$\exists$ un mot de $L(G)$ ayant plusieurs arbres de dérivation distincts (ex. classique : $E \rightarrow E+E \mid E^*E \mid (E) \mid id$).
Grammaire propre	Aucune production $A \rightarrow \epsilon$, et aucune production $A \rightarrow B$ (deux non-terminaux).

2.3 Automate à pile (reconnaisseur associé)

- Théorème : pour toute grammaire algébrique G , il existe un automate à pile (AP) reconnaissant $L(G)$.
- Un AP est défini par $A = \langle \Sigma, \Gamma, \$, E, e_0, F, \Delta \rangle$ (alphabet d'entrée, alphabet de pile, fond de pile, états, état initial, états finaux, relation de transition Δ).
- Le mot est reconnu si on atteint un état final ET la pile est vide en fin de lecture.
- **Méthode** : dérouler le mot lettre par lettre en tenant un tableau Pile / Entrée / État / Transition — vérifier à chaque étape quelle transition Δ (état, sommet de pile, lettre lue) s'applique.

2.4 Hiérarchie de Chomsky (4 types)

Type	Nom	Forme des règles
0	Générale	$\alpha \rightarrow \beta$, avec $\alpha \in (N \cup T)^+$, $\beta \in (N \cup T)^*$
1	Contextuelle	$\alpha \rightarrow \beta$, avec $ \alpha \leq \beta $
2	Hors-contexte (algébrique)	$X \rightarrow \omega$, avec $X \in N$, $\omega \in (N \cup T)^*$
3	Régulière	$X \rightarrow wY$ ou $X \rightarrow w$, avec $X, Y \in N$, $w \in T^*$

Type 0 $\supset$ Type 1 $\supset$ Type 2 $\supset$ Type 3 (chaque type est un cas particulier du précédent). Un langage est « de type x » s'il existe une grammaire de type x qui le génère.

3. Analyse syntaxique

But : étant donné une grammaire et un mot, déterminer si le mot appartient au langage généré, construire l'arbre syntaxique, détecter les erreurs. Deux approches : **descendante** (top-down, de l'axiome vers le mot) et **ascendante** (bottom-up, des feuilles vers la racine).

3.1 Analyse descendante (LL) — méthode complète

Étape 1 — Rendre la grammaire utilisable (si nécessaire)

- Rendre la grammaire propre : supprimer les $A \rightarrow \varepsilon$ (répercuter ε dans les productions où A apparaît) ; si $\varepsilon \in L(G)$, créer un nouvel axiome $S' \rightarrow S \mid \varepsilon$.

- Éliminer la récursivité à gauche — toujours après avoir rendu la grammaire propre.

Élimination de la récursivité immédiate :

$A \rightarrow A\alpha \mid \beta \Rightarrow A \rightarrow \beta A' \text{ et } A' \rightarrow \alpha A' \mid \varepsilon$

Cas général (non-terminaux ordonnés $A_1 \dots A_n$) :

- Pour $i = 1..n$: pour $j = 1..i-1$, remplacer chaque production $A_i \rightarrow A_j \gamma$ (avec $A_j \rightarrow \beta_1 \dots \beta_p$) par $A_i \rightarrow \beta_1 \gamma \mid \dots \mid \beta_p \gamma$; puis éliminer la récursivité immédiate de A_i .

Factoriser à gauche :

Si $A \rightarrow \alpha \beta_1 \mid \dots \mid \alpha \beta_n \mid \gamma_1 \mid \dots \mid \gamma_p$ (α = plus long préfixe commun $\neq \varepsilon$), remplacer par : $A \rightarrow \alpha A' \mid \gamma_1 \mid \dots \mid \gamma_p$ et $A' \rightarrow \beta_1 \mid \dots \mid \beta_n$

Étape 2 — Calculer PREMIERS(X) pour chaque non-terminal X

- Si X est terminal : $\text{PREMIERS}(X) = \{X\}$.
- Si $X \rightarrow \varepsilon$ est une production : ajouter ε à $\text{PREMIERS}(X)$.
- Si $X \rightarrow Y_1 Y_2 \dots Y_n$: ajouter $\text{PREMIERS}(Y_1) \setminus \{\varepsilon\}$ à $\text{PREMIERS}(X)$; si $\varepsilon \in \text{PREMIERS}(Y_1)$, regarder Y_2 , etc. ; si $\varepsilon \in \text{PREMIERS}(Y_i)$ pour tout i, ajouter ε à $\text{PREMIERS}(X)$.
- Répéter jusqu'à stabilisation (point fixe).

Étape 3 — Calculer SUIVANTS(X) pour chaque non-terminal X

- Ajouter \$ à $\text{SUIVANTS}(S)$ (axiome).
- Pour chaque production $X \rightarrow \alpha Y \beta$ (Y non-terminal) : ajouter $\text{PREMIERS}(\beta) \setminus \{\varepsilon\}$ à $\text{SUIVANTS}(Y)$.
- Pour chaque production $X \rightarrow \alpha Y$ (Y en fin de production) : ajouter $\text{SUIVANTS}(X)$ à $\text{SUIVANTS}(Y)$.
- Pour chaque production $X \rightarrow \alpha Y \beta$ avec $\varepsilon \in \text{PREMIERS}(\beta)$: ajouter $\text{SUIVANTS}(X)$ à $\text{SUIVANTS}(Y)$.
- Répéter jusqu'à stabilisation.

Étape 4 — Construire la table d'analyse M[X, a]

- Pour chaque règle $X \rightarrow \omega$: pour tout $a \in \text{PREMIERS}(\omega) \setminus \{\varepsilon\}$, mettre « $X \rightarrow \omega$ » dans $M[X, a]$.
- Si $\varepsilon \in \text{PREMIERS}(\omega)$: pour tout $b \in \text{SUIVANTS}(X)$, mettre « $X \rightarrow \omega$ » dans $M[X, b]$.
- **Grammaire LL(1)** si et seulement si aucune case de la table ne contient plus d'une production.
- Propriétés utiles : toute grammaire LL(1) est nécessairement factorisée à gauche ET non récursive à gauche (mais la réciproque est fausse !).

Étape 5 — Faire fonctionner l'analyseur à pile LL(1)

Répéter : soit X le sommet de pile, a la lettre pointée en entrée.

- X non-terminal : si $M[X, a] = X \rightarrow Y_1 \dots Y_n$, dépiler X, empiler $Y_n \dots Y_1$ (Y_1 en haut), émettre $X \rightarrow Y_1 \dots Y_n$; sinon ERREUR (case vide).

- X = \$: si a = \$, ACCEPTER, sinon ERREUR.

- X terminal : si X = a, dépiler X et avancer dans l'entrée ; sinon ERREUR.

Fin : jusqu'à ACCEPTER ou ERREUR. On construit l'arbre syntaxique en parallèle, à chaque émission de production.

Remarque : certaines grammaires ne sont jamais LL(k) quel que soit k (ex. $S \rightarrow aSb \mid bSa \mid \varepsilon$).

3.1 bis Analyse ascendante (bottom-up) — principe

- On part du mot en entrée et on construit l'arbre des feuilles vers la racine, en « réduisant » des morceaux du mot courant vers un non-terminal (opération inverse de la dérivation).
- Notions clés (à développer en 3.3 du cours) : collection des items, table d'analyse, grammaire SLR(1).

Checklist méthode — face à un exercice

« Le mot w appartient-il à $L(G)$? »

- Descendant : chercher une suite de dérivations gauches (ou droites) partant de S et aboutissant à w .
- Ascendant : partir de w et tenter de « réduire » en remontant jusqu'à S .

« Construire la table LL(1) de G »

- 1) Rendre G propre $\rightarrow$ 2) éliminer récursivité à gauche $\rightarrow$ 3) factoriser à gauche $\rightarrow$ 4) calculer PREMIERS(X) $\rightarrow$ 5) calculer SUIVANTS(X) $\rightarrow$ 6) remplir $M[X,a]$ $\rightarrow$ 7) vérifier LL(1) (au plus 1 règle/case).

« G est-elle réduite / ambiguë / propre / récursive à gauche ? »

- Réduite : tester accessibilité (depuis S) et productivité (vers un mot de T^*) de chaque non-terminal.
- Ambiguë : chercher un mot avec ≥ 2 arbres de dérivation distincts (souvent via opérateurs $+$, $*$ sans priorité).
- Propre : chercher une règle $A \rightarrow \varepsilon$ ou $A \rightarrow B$ (2 non-terminaux).
- Récursive à gauche : chercher $A \rightarrow^+ A\beta$ (immédiate si $A \rightarrow A\beta$ est une règle directe).

« Simuler l'automate à pile / l'analyseur LL(1) sur un mot »

- Toujours tenir un tableau à 3 colonnes : Pile | Entrée restante | (Sortie = production appliquée).
- Vérifier l'état de la pile ($\$$ en bas) et s'arrêter sur ACCEPTER (pile et entrée réduites à $\$$) ou ERREUR.